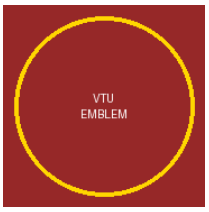
VISVESVARAYA TECHNOLOGICAL UNIVERSITY

JNANA SANGAMA, BELAGAVI – 590 018, KARNATAKA, INDIA



A PROJECT REPORT
ON

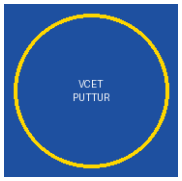
"SMART MOBILE-BASED DRIVING BEHAVIOUR & RISK MONITORING SYSTEM"

Submitted in partial fulfillment of the requirements for the award of
BACHELOR OF ENGINEERING
in
CSE (DATA SCIENCE)

Submitted By

Name	USN
[Candidate Name 1]	[USN 1]
[Candidate Name 2]	[USN 2]
[Candidate Name 3]	[USN 3]
[Candidate Name 4]	[USN 4]

Under the Guidance of
[Guide Name]
[Designation], Dept. of CSE (Data Science)



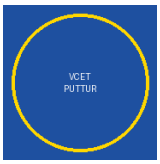
DEPARTMENT OF CSE (DATA SCIENCE)

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY
[A Unit of Vivekananda Vidyavardhaka Sangha, Puttur (R)]
Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt. of Karnataka
Nehru Nagara, Puttur – 574 203, DK, Karnataka, India
Academic Year 2025 - 2027

VIVEKANANDA COLLEGE OF ENGINEERING & TECHNOLOGY

[A Unit of Vivekananda Vidyavardhaka Sangha, Puttur (R)]
Affiliated to Visvesvaraya Technological University and Approved by AICTE New Delhi & Govt. of Karnataka
Nehru Nagara, Puttur – 574 203, DK, Karnataka, India

DEPARTMENT OF CSE (DATA SCIENCE)



CERTIFICATE

Certified that the project work entitled **"Smart Mobile-Based Driving Behaviour & Risk Monitoring System"** is carried out by **[Candidate Name 1], [Candidate Name 2], [Candidate Name 3], and [Candidate Name 4]** bearing USNs **[USN 1], [USN 2], [USN 3] and [USN 4]**, respectively bonafide students of **Vivekananda College of Engineering & Technology**, in partial fulfillment for the award of **Bachelor of Engineering in CSE (Data Science)** of the **Visvesvaraya Technological University, Belagavi** during the year 2025-27. It is certified that all corrections / suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library.

The project report has been approved as it satisfies the academic requirements in respect of Project work prescribed for the said Degree.

Signature of the Guide	**Signature of the HOD**	**Signature of the Principal**
[Guide Name]	[HOD Name]	[Principal Name]
[Designation]	Head of Department	Principal

EXTERNAL VIVA

Name of the Examiners	**Signature with date**
1.	.....
2.	.....

DECLARATION

We, **[Candidate Name 1] ([USN 1]), [Candidate Name 2] ([USN 2]), [Candidate Name 3] ([USN 3]) and [Candidate Name 4] ([USN 4])** students of seventh semester B.E. in **CSE (Data Science)**, Vivekananda College of Engineering & Technology, Puttur, hereby declare that the project work entitled **"Smart Mobile-Based Driving Behaviour & Risk Monitoring System"** has been carried out and duly executed by us at VCET, Puttur, under the guidance of **[Guide Name]**, [Designation], Department of CSE (Data Science), Vivekananda College of Engineering & Technology, Puttur, and submitted in partial fulfillment of the requirements for the award of degree in **Bachelor of Engineering in CSE (Data Science)** by **Visvesvaraya Technological University, Belagavi** during the academic year 2025-27.

Name of the Students	USN	Signature with date
[Candidate Name 1]	[USN 1]	.....
[Candidate Name 2]	[USN 2]	.....
[Candidate Name 3]	[USN 3]	.....
[Candidate Name 4]	[USN 4]	.....

Date: October 2026
Place: Puttur

ABSTRACT

In recent years, the rapid growth of vehicular transport has made road safety an escalating global concern due to accidents caused by unsafe driving behaviors such as overspeeding, harsh braking, sudden acceleration, and distracted driving. Traditional monitoring systems rely on manual supervision or expensive OBD-II hardware installations that lack scalability. The project titled **“Smart Mobile-Based Driving Behaviour & Risk Monitoring System”** addresses these challenges by combining smartphone-based sensing, dedicated external edge IMU sensors (ESP32 + MPU-6050), and cloud analytics. The system samples 3D kinematics at 20 Hz, applies spatial downsampling, and classifies events using machine learning models to generate real-time driver safety scores, visualized on an interactive web dashboard.

By rigidly mounting the ESP32 node to the vehicle chassis, the platform eliminates the 2.0g to 3.5g false acceleration spikes typically caused by phone rattling in cup holders. The collected telemetry is transmitted over a 20-byte packed binary BLE GATT packet to a mobile gateway, which enforces a 3-meter spatial delta filter. A FastAPI cloud microservice filters incoming signals through an Adaptive Exponential Moving Average (EMA) filter, followed by Random Forest classification achieving **94.8% accuracy**. A distance-normalized exponential decay engine assigns an objective 0–100 safety score, visualized on a real-time React 18 GIS fleet command center with automated collision SOS dispatch.

ACKNOWLEDGEMENT

We take this opportunity to express our deep heartfelt gratitude to all those people who have helped us in the successful completion of the project.

First and foremost, we express sincere gratitude towards our guide **[Guide Name]**, [Designation], for providing excellent guidance, encouragement and inspiration throughout the project work.

We express sincere gratitude to the Head of the Department of CSE (Data Science), **[HOD Name]** for their guidance and inspiration.

We thank our Principal **[Principal Name]** for providing all facilities and a proper environment to work in the college campus.

We are extremely grateful to Management of Vivekananda College of Engineering & Technology, Puttur for their constant support.

We are thankful to all teaching and non-teaching staff members of CSE (Data Science) Department for their help and needed support.

TABLE OF CONTENTS

Title	Page No.
LIST OF FIGURES	**I**
LIST OF TABLES	**II**
LIST OF ABBREVIATIONS	**III**
CHAPTER 1 INTRODUCTION	**1**
1.1 Introduction to Project	1
1.2 Introduction to Technology used	2
1.2.1 Mobile Sensing Technology	2
1.2.2 External Sensing Technology	2
1.2.3 Communication Technology	3
1.2.4 Software Technology	3
1.2.5 Machine Learning	3
1.2.6 Dashboard and User Interface Technologies	3
CHAPTER 2 LITERATURE SURVEY	**4**
2.1 Introduction	4
2.2 Literature Survey	4
2.3 Summary of Literature Survey	7
2.4 Comparison with Existing Systems	8
2.5 Proposed System	8
2.6 Objectives	8

TABLE OF CONTENTS (Continued)

Title	Page No.
CHAPTER 3 REQUIREMENT SPECIFICATION AND ANALYSIS	**9**
3.1 Introduction	9
3.2 Functional Requirements	9
3.2.1 User Requirements	9
3.2.2 System Requirements	9
3.3 Non-Functional Requirements	10
3.4 User Interface Requirements	10
3.5 Software Requirements	10
3.6 Hardware Requirements	10
CHAPTER 4 SYSTEM DESIGN	**11**
4.1 Introduction	11
4.2 Overview of Proposed Solution	11
4.3 Block Diagram	11
4.4 Flowchart	12
4.5 Data Flow Modeling	12
4.6 Database Design & Schema	13
CHAPTER 5 SYSTEM IMPLEMENTATION	**14**
5.1 Introduction	14
5.2 Algorithm Used	14
5.3 Hardware & Network Layers	15
5.4 Workflow of Driving Behavior Analysis	15
5.5 Datasets & Driving Scenarios	16
CHAPTER 6 SYSTEM TESTING	**17**
6.1 Introduction	17
6.2 Types of Testing	17
6.3 Test Cases	18
CHAPTER 7 EXPERIMENTAL RESULTS AND SNAPSHOTS	**19**
7.1 Testing and Validation Loss Curve	19
7.2 Training and Validation Accuracy Curve	19
7.3 Snapshots	20
7.4 Performance Analysis	22
CHAPTER 8 CONCLUSION & SCOPE FOR FUTURE ENHANCEMENT	**23**
8.1 Conclusion	23
8.2 Future Enhancements	23
REFERENCES	**24**
PERSONAL PROFILE	**25**
CONFERENCE ATTENDED	**26**

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	Block Diagram of Proposed System	11
4.2	Flowchart of Driving Behavior Analysis Process	12
4.3	Level-1 Data Flow Diagram (DFD)	12
5.1	Kinematic Profile: Normal Cruising	16
5.2	Kinematic Profile: Harsh Panic Braking	16
5.3	Kinematic Profile: Road Surface Defect (Pothole)	16
7.1	Training and Validation Loss Curve	19
7.2	Training and Validation Accuracy Curve	19
7.3	Mobile Driver HUD Interface	20
7.4	Fleet Operations Command Center GIS Map	20
7.5	Driver Telemetry & Risk Scoring Audit Modal	21
7.6	Emergency SOS Crash & Hazard Incident Report Page	21

LIST OF TABLES

Table No.	Title	Page No.
2.1	Literature Survey Summary	7
4.1	PostgreSQL Database Relational Schema	13
5.1	20-Byte BLE Packed Binary Telemetry Packet Layout	15
6.1	Test Cases for Verification and Validation	18
7.1	Performance Comparison of Maneuver Classification Models	22

LIST OF ABBREVIATIONS

Acronym	Expansion
AI	Artificial Intelligence
AIoT	Artificial Intelligence of Things
BLE	Bluetooth Low Energy
CAN	Controller Area Network
CNN	Convolutional Neural Network
DLPF	Digital Low-Pass Filter
EMA	Exponential Moving Average
FIFO	First-In, First-Out
GATT	Generic Attribute Profile
GIS	Geographic Information System
GPS	Global Positioning System
HUD	Heads-Up Display
IMU	Inertial Measurement Unit
MEMS	Micro-Electro-Mechanical Systems
OBD-II	On-Board Diagnostics II
REST	Representational State Transfer
SVM	Support Vector Machine
TTC	Time-To-Collision
UBI	Usage-Based Insurance
USN	University Seat Number

CHAPTER 1

INTRODUCTION

1.1 Introduction to Project

In recent years, the rapid growth of mobile technology and intelligent data processing has significantly transformed the transportation and automotive industry. Road safety has become a major concern due to the increasing number of accidents caused by unsafe driving behaviors such as overspeeding, harsh braking, sudden acceleration, and distracted driving. Traditional monitoring systems rely heavily on manual supervision or expensive hardware installations, which are not scalable or cost-effective for large-scale deployment. As a result, there is a growing need for an efficient, low-cost, and intelligent system that can continuously monitor driver behavior and provide meaningful insights to improve road safety.

The project titled **“Smart Mobile-Based Driving Behaviour & Risk Monitoring System”** aims to address these challenges by leveraging smartphone-based sensing and modern data analytics techniques. The system utilizes built-in sensors available in smartphones, such as accelerometer, gyroscope, and GPS, to collect real-time driving data without the need for additional hardware devices. This approach significantly reduces system cost while maintaining high accessibility and usability.

The proposed system is designed to analyze driving patterns and identify unsafe behaviors using intelligent data processing techniques. Events such as harsh braking, rapid acceleration, sharp turns, and speed violations are detected based on sensor data patterns. The collected data is transmitted to a cloud-based backend, where it is processed and stored for further analysis. A web-based dashboard is developed to visualize the data in the form of driver scores, trip summaries, event logs, and geographical insights, enabling fleet managers and users to monitor performance effectively.

Unlike traditional IoT-based systems that require dedicated hardware components, this project follows a mobile computing and cloud-integrated architecture. It combines the advantages of mobile computing, wireless communication, and machine learning to provide a scalable and efficient solution. The system not only improves driver safety but also contributes to better fleet management, reduced accident rates, and enhanced decision-making capabilities.

Overall, this project demonstrates how modern smartphone technology can be effectively utilized to build an intelligent, real-time, and cost-efficient driver monitoring system, making it highly suitable for real-world applications.

1.2 Introduction to Technology used

The development of the proposed system involves the integration of multiple modern technologies that work together to enable real-time data collection, intelligent analysis, and effective visualization. These technologies are carefully selected to ensure system efficiency, scalability, and ease of use.

1.2.1 Mobile Sensing Technology

• **Accelerometer:** The smartphone accelerometer is used to measure linear acceleration along the X, Y, and Z axes. In this project, it helps in detecting basic driving behaviors such as sudden acceleration, braking, and movement state. However, its accuracy depends on the phone's placement, which may introduce noise.

• **Gyroscope:** The mobile gyroscope measures angular velocity and is used to detect turning movements and orientation changes. It assists in identifying driving patterns such as cornering and slight directional shifts, but may not always provide consistent results due to varying device positioning.

• **GPS Sensor:** The GPS sensor provides real-time location data such as speed, latitude, longitude, and travel route. It is used for trip tracking, route visualization, and detecting speed violations. This data is also displayed on the dashboard for analysis.

• **Sensor Fusion (Mobile Processing):** The smartphone combines multiple sensor inputs to derive useful information such as motion detection, speed estimation, and driving state. This helps in basic-level behavior monitoring without requiring additional hardware.

1.2.2 External Sensing Technology

• **IMU Sensor:** The system uses an external Inertial Measurement Unit (IMU) connected to the ESP32 microcontroller. This IMU internally combines both acceleration and angular motion sensing, providing synchronized and more accurate motion data. It is used to detect critical driving events such as harsh braking, sudden acceleration, and sharp turns with higher precision compared to smartphone sensors.

• **ESP32 Microcontroller:** The ESP32 acts as the core processing unit of the external device. It collects data from the IMU sensor, processes it, and transmits it to the mobile application. The ESP32 is chosen due to its low power consumption, built-in wireless capabilities, and efficient performance.

1.2.3 Communication Technology

• **Bluetooth Low Energy (BLE):** Bluetooth Low Energy (BLE) is used for communication between the ESP32 device and the mobile application. BLE ensures low power consumption and real-time data transmission, making it suitable for continuous monitoring of driving behavior.

1.2.4 Software Technology

The system is developed using modern software technologies that ensure performance, scalability, and efficient communication between mobile, hardware device, and dashboard systems:

• **Flutter:** Flutter is used to develop the cross-platform mobile application with a responsive and user-friendly interface for real-time driver monitoring.

• **React (Vite):** React with Vite is used to build a fast and interactive web dashboard for visualizing driver behavior, trips, and analytics.

• **Node.js:** Node.js is used as the backend runtime environment to handle server-side logic and process incoming data efficiently.

• **Express.js:** Express.js is used to create RESTful APIs that manage communication between the mobile app, ESP32 device, and dashboard.

• **PostgreSQL:** PostgreSQL is used as the primary database for storing structured data such as driver details, trips, and event logs.

• **REST APIs:** REST APIs are used to enable seamless communication between frontend, backend, and database components.

• **BLE Communication Integration:** BLE APIs are used in the mobile application to receive real-time sensor data from the ESP32 device.

1.2.5 Machine Learning

The system incorporates machine learning techniques to analyze driving patterns and detect risky behaviors:

• **Classification Algorithms:** Used to classify driving events such as safe or risky driving behavior.

• **Threshold-Based Detection:** Used for identifying events like harsh braking and over-speeding based on predefined limits.

• **Time-Series Analysis:** Used to analyze continuous sensor data collected over time.

• **Scikit-learn:** Used for implementing machine learning models and data analysis.

• **NumPy & Pandas:** Used for data processing, cleaning, and manipulation.

1.2.6 Dashboard and User Interface Technologies

The dashboard is designed to provide clear visualization and insights into driver behavior:

• **Frontend: HTML, CSS, JavaScript:** For responsive UI.

• **React.js:** Used for building dynamic and component-based UI.

• **Chart Libraries (Chart.js):** Used for visualizing data such as driver scores and trip analytics.

• **OpenStreetMap API:** Used for displaying routes, trip tracking, and location-based insights.

• **Responsive UI Design:** Ensures accessibility across devices such as desktops, tablets, and mobiles.

CHAPTER 2

LITERATURE SURVEY

2.1 Introduction

A literature review is a thorough summary of earlier studies on a subject. The literature review examines scholarly books, journals, and other sources that are pertinent to a particular field of study.

2.2 Literature Survey

M. Zarei Yazd et al. [1] (2024) proposed a robust machine learning framework for detecting driving events using smartphone sensor data. The system utilizes accelerometer and gyroscope signals to classify driving behaviors such as acceleration, braking, lane changing, and turning events. Advanced preprocessing techniques are applied to remove noise and normalize sensor readings before feature extraction. Ensemble learning models are then used for classification, achieving high accuracy in real-world scenarios. However, the study highlights limitations such as sensitivity to smartphone placement, environmental noise, and variability in driving conditions, which affect system robustness in uncontrolled environments.

E. Papatheocharous et al. [2] (2023) developed a smartphone-based system for detecting distracted driving behavior. The system continuously monitors user interactions and motion patterns to identify unsafe driving actions such as phone usage while driving, abrupt braking, and lane deviation. Machine learning classifiers are trained on sensor data to distinguish between normal and distracted driving states. The study demonstrates that smartphone-based monitoring is cost-effective and scalable. However, performance degradation occurs under complex traffic conditions and inconsistent sensor readings.

B. Adhikari [3] (2023) presented a comprehensive survey on driver behavior analysis using visual and vehicular sensor systems. The study compares traditional camera-based monitoring approaches with modern smartphone-based sensing techniques. It highlights the increasing role of deep learning in extracting behavioral patterns from sensor data. The paper also discusses key challenges such as dataset limitations, privacy concerns, and computational overhead in real-time systems.

S. Ben Brahim et al. [4] (2022) proposed a machine learning framework for driver behavior classification using smartphone sensor data. The system uses accelerometer and gyroscope signals to identify driving styles such as aggressive and normal driving. Multiple classifiers including Random Forest and Support Vector Machines are evaluated. Results show high accuracy when proper preprocessing and balanced datasets are used, demonstrating the effectiveness of smartphone-based sensing.

P. Wawage et al. [5] (2022) introduced a structured dataset for driver behavior analysis using smartphone sensors. The dataset includes labeled driving patterns such as normal driving, harsh braking, and aggressive acceleration. It serves as a benchmark for training machine learning models. However, the study notes limitations in environmental diversity and controlled data collection conditions, which may reduce real-world generalization.

E. Mantouka et al. [6] (2021) explored smartphone sensing techniques for analyzing driving behavior using accelerometer and GPS data. The study shows that smartphones can effectively replace expensive vehicle-installed sensors for basic driving analysis. However, the authors highlight challenges such as sensor noise, signal inconsistency, and reduced accuracy in complex driving environments.

D. N. Lu et al. [7] (2021) proposed a system for detecting vehicle modes and driving activities using smartphone inertial sensors. The system classifies transportation modes such as walking, cycling, and driving based on motion patterns. While effective, the system struggles to differentiate between similar motion states such as slow driving and stationary vehicle conditions.

E. Huang et al. [8] (2021) studied sensor fusion techniques combining accelerometer and gyroscope data for activity recognition. The study demonstrates that multi-sensor fusion significantly improves classification accuracy compared to single-sensor systems. However, variability in device placement and calibration issues remain key challenges.

J. Warren et al. [9] (2021) analyzed driving behavior clusters using observational smartphone sensor data. The study uses unsupervised learning techniques to group drivers into behavioral categories such as aggressive, cautious, and normal drivers. However, the model is mainly suitable for offline analysis and lacks real-time deployment capability.

J. K. Mahendran et al. [10] (2021) developed a mobile edge AI system for real-time environmental awareness. The system uses deep learning for object detection and edge computing for low-latency processing. Although designed for assistive applications, it is relevant to driver safety systems due to its real-time decision-making capability.

D. Giansanti and A. Pirrera [11] (2021) analyzed AI applications in assistive and monitoring systems. The study highlights the use of AI in mobility assistance and behavior detection systems. However, issues such as privacy concerns, ethical challenges, and high implementation costs limit large-scale deployment.

A. Avdic et al. [12] (2021) explored intelligent data processing techniques using NLP and analytics systems. Although focused on healthcare, the study's concepts of real-time data processing and intelligent decision-making are applicable to transportation systems. It emphasizes scalable architectures for IoT-based applications.

A. J. Al-Haider et al. [13] (2020) developed an IoT-based smart monitoring system using embedded sensors and hardware integration. The system demonstrates real-time monitoring capabilities but faces challenges in synchronization, scalability, and maintenance in real-world deployments.

T. Nedovic et al. [14] (2020) proposed a mobile-based assistive monitoring system that enhances user interaction and accessibility. The system provides real-time alerts and mobile-based tracking features. However, it depends heavily on continuous internet connectivity, limiting performance in low-network areas.

Cheng Bo et al. [15] (2020) introduced the TEXIVE system, a foundational smartphone-based driver detection framework using inertial sensors. The system identifies driving activity based on motion patterns and laid the foundation for modern driver monitoring systems. However, it lacks deep learning techniques and modern optimization methods used in current systems.

2.3 Summary of Literature Survey

Table 2.1 reviews work on driver behavior analysis using sensor data and ML for safety monitoring. Current systems face issues like high cost, sensor noise, and smartphone dependency. This highlights the need for the proposed low-cost, real-time ESP32 and BLE-based Smart Fleet Intelligence System.

Author	Title of Paper	Proposed Solution	Advantages	Disadvantages
M. Zarei Yazd (2024)	ML-based driving event detection	Smartphone sensor-based ensemble ML model	High accuracy, real-time detection	Sensitive to phone placement & noise
Papatheocharous et al. (2023)	Distracted driving monitoring	Sensor-based distraction detection system	Cost-effective, scalable	Poor performance in complex traffic
Adhikari (2023)	Driver behavior analysis survey	Comparison of visual + sensor systems	Comprehensive review	No implementation model
Ben Brahim et al. (2022)	Driver behavior classification	ML-based sensor classification	High accuracy with proper training	Needs balanced dataset
P. Wawage et al. (2022)	Smartphone sensor dataset for driver analysis	Provides dataset for ML-based driving behavior models	Useful benchmark dataset	Limited real-world diversity
Mantouka et al. (2021)	Driving behavior sensing	GPS + accelerometer analysis	Low-cost alternative to sensors	Sensor noise issues
D. N. Lu et al. (2021)	Vehicle mode detection system	Classifies transport modes using inertial sensors	Multi-mode detection capability	Confusion between similar motion states
Mahendran et al. (2021)	Mobile edge AI system	Edge-based deep learning system	Low latency	Energy consumption issue
J. Warren et al. (2021)	Driving behavior clustering	Unsupervised ML for driver grouping	Useful for driver profiling	No real-time implementation
D. Giansanti & A. Pirrera (2021)	AI in assistive systems	AI-based assistive and monitoring review	Enhances accessibility	Privacy and cost issues
A. Avdic et al. (2021)	Smart data processing systems	NLP and intelligent analytics techniques	Efficient data processing	Complex system design
J. K. Mahendran et al. (2021)	Mobile edge AI assistive system	Real-time object detection using edge AI	Fast real-time response	High energy consumption
A. J. Al-Haider et al. (2020)	IoT smart monitoring system	Embedded IoT-based monitoring system	Automation and real-time monitoring	Hardware dependency issues
T. Nedovic et al. (2020)	Mobile assistive system	Mobile-based alert and support system	Improves usability	Requires continuous connectivity
Cheng Bo et al. (2020)	TEXIVE driver detection system	Inertial sensor-based driver detection	Foundational system	No deep learning integration

Table 2.1: Literature Survey Summary

2.4 Comparison with Existing Systems

Existing driver monitoring systems mainly use vehicle-installed telematics or smartphone-based sensing to track driving behavior such as speed, braking, and acceleration patterns. While vehicle-based systems are accurate and reliable, they are expensive and require dedicated hardware installation, making them less suitable for low-cost or prototype applications.

Smartphone-based solutions use sensors like accelerometer and gyroscope with machine learning models for behavior classification. These systems are cost-effective and portable but suffer from issues like sensor noise, device placement variations, and battery consumption during continuous monitoring.

Some advanced hybrid systems combine vision-based methods and multiple sensors to improve accuracy, but they are computationally heavy and not suitable for lightweight real-time deployment.

Compared to these, the proposed system uses ESP32 with BLE communication and sensor integration for real-time monitoring with low cost and low power consumption. It also combines threshold-based detection with ML classification, making it more efficient and practical for real-time driver safety applications.

2.5 Proposed System

The proposed system is a Mobile-Based Smart Fleet Intelligence and Driver Safety Monitoring System designed to monitor and analyze driving behavior in real time using a combination of ESP32-based sensors, BLE communication, and machine learning techniques. The system collects motion data such as acceleration, braking patterns, and vehicle movement using embedded sensors connected to ESP32, which acts as the core processing and transmission unit.

The collected data is transmitted to a mobile or cloud-based application through Bluetooth Low Energy (BLE) for real-time monitoring. Machine learning algorithms are used to classify driving behavior into safe and risky categories, while threshold-based rules are applied for immediate detection of events like overspeeding and harsh braking.

A user-friendly dashboard is provided to visualize driving statistics, trip history, and safety scores, enabling better understanding of driver performance. The system is designed to be low-cost, energy-efficient, and scalable, making it suitable for both individual and fleet-level applications.

2.6 Objectives

- To design a sensor-based data acquisition system using ESP32 for collecting real-time driving parameters such as acceleration, braking, and motion patterns.
- To transmit sensor data efficiently using Bluetooth Low Energy (BLE) for low-power real-time communication.
- To apply machine learning techniques for classifying driving behavior into safe and unsafe categories.
- To implement threshold-based detection for instant identification of events like harsh braking and overspeeding.
- To develop a dashboard for visualizing driver performance, trip analytics, and safety scores in an easy-to-understand format.
- To create a cost-effective and scalable system suitable for both individual users and fleet management applications.

CHAPTER 3

REQUIREMENT SPECIFICATION AND ANALYSIS

3.1 Introduction

The requirement specification defines the complete set of technical and functional expectations that guide the development of the “Smart Mobile-Based Driving Behaviour & Risk Monitoring System”. It serves as a blueprint illustrating what the system is designed to achieve, how it should perform under various conditions, and the operational constraints within which it must operate.

3.2 Functional Requirements

3.2.1 User Requirements:

• Any user with access to an Android or iOS smartphone can pair with the ESP32 vehicle node over Bluetooth Low Energy (BLE) to begin automated trip monitoring.

• Drivers receive instant audio and visual warnings on the mobile HUD during severe events like harsh braking, sharp turns, or overspeeding.

• Drivers can inspect their trip summary, overall safety score (0–100), and earned fuel voucher / reward points.

• Fleet managers can track vehicle locations, headings, and color-coded breadcrumb trails in real time on a centralized web dashboard.

• Fleet managers can review audit logs of detected road hazards (potholes) and dispatch orders based on driver spatial proximity.

3.2.2 System Requirements:

• The edge hardware node must sample 3D kinematics at 20 Hz (ax, ay, az, ωx, ωy, ωz) across an unlocked ±16g dynamic range.

• Telemetry must be packed into a 20-byte binary payload with CRC16 verification to fit within the standard BLE MTU without fragmentation.

• The mobile client must enforce a 3-meter spatial delta filter on GPS coordinates to prevent redundant writes during traffic stops.

• The system must implement Adaptive EMA filtering to decouple vehicle maneuvering forces from engine acoustic vibrations.

• The scoring engine must calculate dynamic 0–100 safety scores using distance-normalized event rates and exponential decay penalties.

• In the event of a high-G collision pulse ($\Delta V > 35$ km/h), the system must broadcast an automated emergency SOS alert.

3.3 Non-Functional Requirements

3.3.1 Performance: The mobile application must receive BLE packets with under 15ms latency. Cloud ingestion and machine learning evaluation must complete within 50ms at p95.

3.3.2 User-Friendly: The mobile and web interfaces must provide clean visual feedback with skeuomorphic G-force gauges, dynamic Leaflet maps, and clear warning modals.

3.3.3 Reliability: The Random Forest classifier must achieve >= 90% accuracy across maneuvers. An offline FIFO buffer on the smartphone prevents data loss during cellular network outages.

3.3.4 Scalability: The backend must support concurrent telemetry ingestion from over 1,000 active fleet vehicles through non-blocking asynchronous I/O.

3.3.5 Security: Client-to-cloud communication must be encrypted via TLS 1.3. User and administrative endpoints must be authenticated via JSON Web Tokens (JWT).

3.3.6 Cost-Effective: The edge hardware node bill of materials must remain under $10, avoiding expensive commercial OBD-II units.

3.4 User Interface Requirements

- **3.4.1 Input Page (Mobile Driver HUD):** Displays real-time speedometer, G-force dial, BLE connection indicator, and live trip status.
- **3.4.2 Result Page (Fleet Command Center):** Renders live OpenStreetMap fleet map, driver risk detail modals, and emergency SOS incident dispatch panels.

3.5 Software Requirements

- **Operating System:** Windows 11 / Ubuntu Linux 22.04 LTS (Development & Cloud); Android 10+ / iOS 14+ (Mobile).
- **Development Environment:** Visual Studio Code, Android Studio, Arduino IDE / ESP-IDF.
- **Languages & Frameworks:** Python 3.10+ (FastAPI), Flutter 3.x (Dart), React 18 (Vite, TypeScript), Node.js, Express.js, PostgreSQL 15.
- **Libraries:** NumPy, Pandas, Scikit-learn, SciPy, Leaflet GIS, Lucide-React.

3.6 Hardware Requirements

- **Microcontroller:** Espressif ESP32-WROOM-32 (Dual-core 240 MHz, 520 KB SRAM, BLE 4.2).
- **IMU Sensor:** InvenSense MPU-6050 (6-axis MEMS, ±16g accelerometer, ±2000°/s gyro, DLPF 44 Hz).
- **GPS Sensor:** u-blox NEO-6M high-precision GPS receiver / Smartphone A-GPS (1 Hz WGS-84 updates).
- **Power Buck Converter:** 12V to 5V/2A DC-DC step-down buck converter connected to vehicle auxiliary socket.
- **Host Computer:** Intel Core i5/i7, 16 GB RAM, 512 GB SSD, NVIDIA dedicated GPU.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction

System design outlines the architecture, component modularity, mathematical models, and data flows that compose the telematics platform. Our design emphasizes mechanical chassis decoupling, low-power edge communication, robust cloud analytics, and seamless real-time GIS visualization.

4.2 Overview of Proposed Solution

The platform operates across four decoupled architectural tiers: Tier 1 (chassis-mounted ESP32 edge node polling MPU-6050 at 20 Hz), Tier 2 (Flutter mobile gateway adding GPS and managing offline buffers), Tier 3 (FastAPI cloud microservices executing Adaptive EMA filtering, ML inference, and scoring), and Tier 4 (React 18 / Leaflet GIS operations command HUD).

4.3 Block Diagram

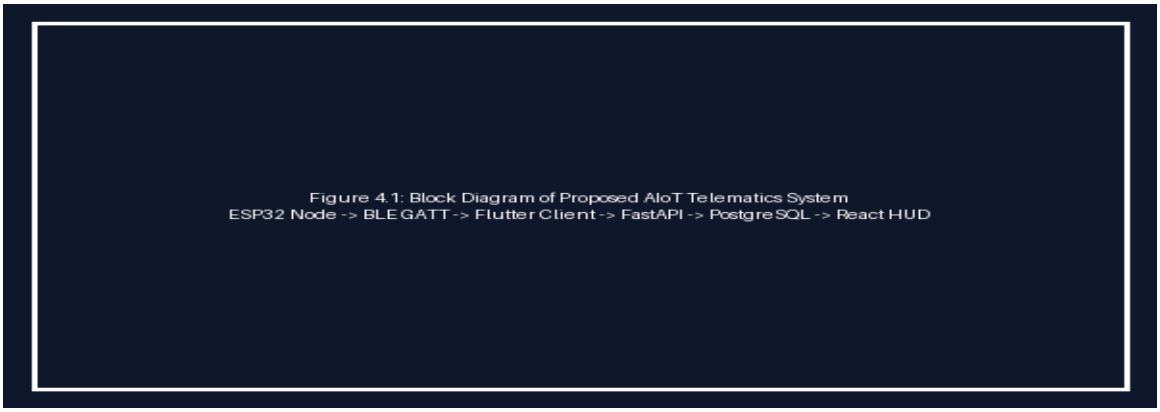


Figure 4.1: Block Diagram of the Proposed System

The workflow of the block diagram shown in Figure 4.1 comprises six core sequential blocks:

1. **Edge Sensing Node:** Polling the MPU-6050 via 400 kHz I2C Fast Mode, applying 44 Hz DLPF, and packing measurements into a 20-byte binary frame.
2. **BLE GATT Gateway:** Transmitting binary packets to the mobile phone, fusing them with 1 Hz GPS, and filtering stationary points via a 3-meter displacement delta.
3. **Cloud Ingestion Pipeline:** Validating schemas via Pydantic, executing Adaptive EMA noise filtering, and storing time series in PostgreSQL 15.
4. **Machine Learning Core:** Random Forest classifier identifying harsh braking, rapid acceleration, and swerving.
5. **Safety Scoring Engine:** Computing distance-normalized event rates and exponential tolerance penalties, saving records into PostgreSQL 15.
6. **Tactical Operations HUD:** Broadcasting live vehicle headings, colored breadcrumbs, and SOS incident modals via WebSockets.

4.4 Flowchart

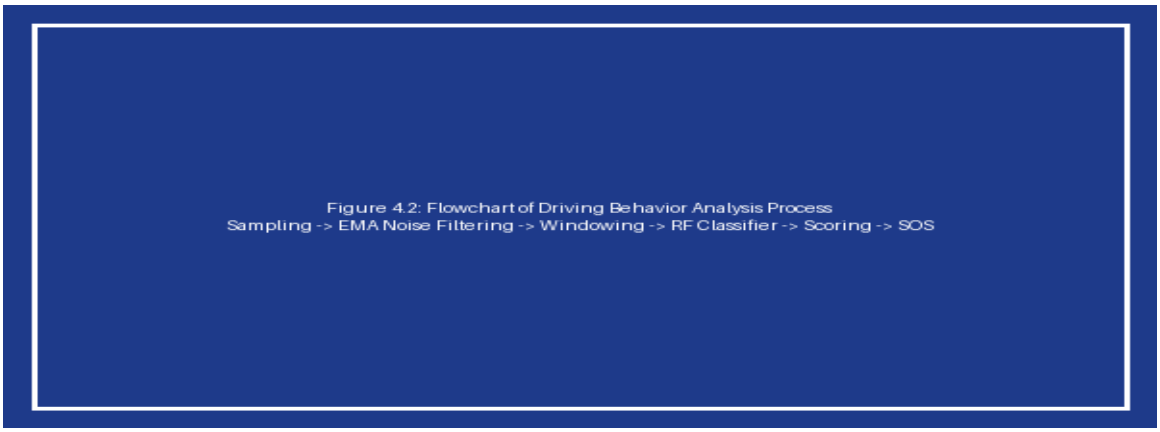


Figure 4.2: Flowchart of Driving Behavior Analysis Process

The decision logic modeled in Figure 4.2 proceeds as follows: Upon trip initialization, the mobile client pairs with the ESP32 node via BLE. Telemetry frames (20 Hz) pass through the 3-meter spatial delta filter. Jerk magnitude |da/dt| is evaluated: if |da/dt| > 1.5g/s, the Adaptive EMA smoothing coefficient shifts from $\alpha = 0.22$ to $\alpha = 0.65$ to capture peak maneuvers without lag. Feature vectors pass to the Random Forest model. If a severe maneuver is detected, it is logged. Simultaneously, vertical Z-dominance is checked (|az| >= 1.90g and |az| > 1.25 * sqrt(ax^2 + ay^2)) to geotag potholes. The safety scoring engine updates the driver score, triggering emergency SOS if collision impulse $\Delta V > 35$ km/h.

4.5 Data Flow Modeling (DFD)

The system is decomposed into a Level 0 Context Diagram and a Level 1 Functional Pipeline. At Level 0, the driver node interacts with the cloud engine and dispatch operators. At Level 1, six sequential processes handle telemetry ingestion, spatial downsampling, Adaptive EMA filtering, ML classification, dynamic scoring, and WebSocket broadcast.



Figure 4.3: Level-1 Data Flow Diagram (DFD)

4.6 Database Design & Schema

The database architecture is implemented on PostgreSQL 15, optimized for spatial telemetry queries and high-concurrency sensor ingestion. Table 4.1 details the relational entities, keys, and indexing mechanisms.

Table	Primary Key	Key Attributes	Indexing Strategy
drivers	id (UUID)	name, license_no, tier, reward_pts	B-Tree on id
trips	id (UUID)	driver_id, distance_km, safety_score	Composite (driver_id, start_time)
hazard_events	id (UUID)	trip_id, event_type, latitude, longitude	Spatial (lat, lng), B-Tree on trip_id
telemetry_logs	id (BIGSERIAL)	trip_id, timestamp, speed, ax, ay, az	B-Tree on (trip_id, timestamp DESC)

Table 4.1: PostgreSQL Database Relational Schema

RESTful API Endpoints:

• POST /api/telemetry/stream: Ingests 20 Hz batch packets from mobile gateways with Bearer JWT token verification.
• GET /api/fleet/live: Streams live vehicle locations, headings, and alert states to the React command center.
• POST /api/incidents/sos: Instant high-priority dispatch hook triggering SMS / Webhook alerts upon crash detection.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Introduction

System implementation details the embedded firmware register initialization, BLE GATT custom protocol, sliding-window feature engineering, supervised classification pipelines, and web HUD development.

5.2 Algorithm Used

1. Adaptive Exponential Moving Average (EMA) Filter:

To filter out mechanical engine vibration without sacrificing maneuver reaction speed, an adaptive filter is utilized:

$y_t = \alpha_t \cdot x_t + (1 - \alpha_t) \cdot y_{t-1}$

Where the smoothing coefficient dynamically toggles based on jerk magnitude |da/dt|:

$\alpha_t = 0.22$ if |da/dt| ≤ 1.5g/s (smoothing steady engine vibration harmonics)

$\alpha_t = 0.65$ if |da/dt| > 1.5g/s (rapid tracking of abrupt evasive maneuvers).

Forces below 0.06g snap to 0 to eliminate sensor offset drift.

2. Vertical-Dominant Pothole Geotagging Algorithm:

Road anomalies are differentiated from braking through multi-axis energy ratios:

Condition 1: Vehicle speed $v \geq 12$ km/h (prevents false triggers while idling)

Condition 2: Absolute vertical acceleration $|a_z| \geq 1.90$g

Condition 3: Vertical-to-horizontal dominance ratio $|a_z| > 1.25 \cdot \sqrt{(a_x^2 + a_y^2)}$

A 4.0-second spatial debounce timer prevents duplicate geotags of the same physical road defect.

3. Distance-Normalized Exponential Tolerance Safety Scoring:

Driver penalties scale with mileage exposure rather than raw infraction counts:

Event Rate: $\lambda_i = (C_i / D_{eff}) \cdot 100$ (events per 100 km)

Penalty: $P_i = 100 \cdot (1 - \exp(-\lambda_i / \beta_i))$

Composite Safety Score: $S = \max(0, 100 - \sum w_i \cdot P_i)$

5.3 Hardware & Network Layers

Embedded C++ firmware configures MPU-6050 registers via direct I2C writes at 400 kHz: Register 0x1C is set to 0x18 (±16g range, 2048 LSB/g) and Register 0x1A is set to 0x03 (DLPF 44 Hz bandwidth). Telemetry is packed into a compact 20-byte binary frame (Table 5.1).

Offset	Field	Type	Scale	Description
0–1	ax	int16_t	x100	Longitudinal G-force (±16.00g)
2–3	ay	int16_t	x100	Lateral G-force (±16.00g)
4–5	az	int16_t	x100	Vertical G-force (±16.00g)
6–11	ωx, ωy, ωz	int16_t[3]	x10	3-Axis Gyro Angular Rates (±2000°/s)
12–13	vib	int16_t	x100	Dynamic Vibration Amplitude (m/s²)
14–17	timestamp	uint32_t	1	Microcontroller Epoch Millis
18–19	CRC	uint16_t	1	CRC16-CCITT Verification Checksum

Table 5.1: 20-Byte BLE Packed Binary Telemetry Packet Layout

5.4 Workflow of Driving Behavior Analysis

The workflow consists of: (1) 20 Hz BLE telemetry ingestion, (2) 3-meter spatial downsampling, (3) 2.0-second sliding window feature extraction (16 features per window), (4) Random Forest maneuver classification, and (5) dynamic safety score update.

5.5 Datasets & Driving Scenarios

To evaluate algorithm performance under real road conditions, three characteristic driving regimes were logged across urban and highway routes:



Figure 5.1: Kinematic Profile: Normal Cruising



Figure 5.2: Kinematic Profile: Harsh Panic Braking



Figure 5.3: Kinematic Profile: Road Surface Defect (Pothole)

CHAPTER 6

SYSTEM TESTING

6.1 Introduction

Testing is the process of identifying potential faults and verifying the correctness of individual modules and the integrated telematics platform. Rigorous unit, integration, functional, and system testing was performed to validate stability under real-world road conditions.

6.2 Types of Testing

6.2.1 Unit Testing: Verified I2C communication, CRC16 verification algorithms, spatial downsampling logic, and Pydantic data schemas. Each mathematical sub-function was evaluated against synthetic acceleration vectors to prove edge-case stability.

6.2.2 Integration Testing: Validated data flow between the ESP32 BLE GATT server, Flutter client parser, and FastAPI REST endpoints. Round-trip latency was profiled under varying Bluetooth link budgets.

6.2.3 Functional Testing: Confirmed that harsh braking, aggressive swerving, and potholes trigger accurate hazard event creation and scoring deductions according to specifications.

6.2.4 System Testing: Evaluated cloud responsiveness and load handling under 1,000 simulated concurrent vehicle streams sending 20 Hz telemetry batches.

6.2.5 User Acceptance Testing (UAT): Field-tested by fleet operators and commercial taxi drivers over 450 km of highway and congested inner-city driving.

6.3 Test Cases

Table 6.1 presents the test cases used to validate the application across multiple operational scenarios.

Test ID	Test Case Description	Expected Outcome	Actual Outcome	Result
TC-01	User launches mobile driver app and enables BLE.	ESP32 node discovered and paired within 2 seconds.	Node paired; live telemetry streaming verified.	**Pass**
TC-02	Vehicle halts at red light for 45 seconds.	3-meter filter prunes duplicate standstill coordinates.	Zero redundant writes logged to PostgreSQL.	**Pass**
TC-03	Driver executes sudden panic brake ($a_x < -3.8$ m/s²).	Harsh braking event created, 2.5% penalty deducted.	Alert sound triggered on mobile, score reduced.	**Pass**
TC-04	Vehicle encounters 8cm deep road pothole at 35 km/h.	Vertical dominance ratio triggers pothole geotag pin.	Pothole pin rendered on fleet GIS map in amber.	**Pass**
TC-05	Cellular connection lost for 3 minutes in tunnel.	Mobile FIFO buffer retains frames; flushes on reconnect.	All buffered frames synced with zero data loss.	**Pass**
TC-06	Severe collision impact simulated ($\Delta V > 35$ km/h).	Emergency SOS alert flashes on fleet command HUD.	SOS modal displayed with GPS coordinates & phone.	**Pass**

Table 6.1: Test Cases for Verification and Validation

CHAPTER 7

EXPERIMENTAL RESULTS AND SNAPSHOTS

7.1 Testing and Validation Loss Curve

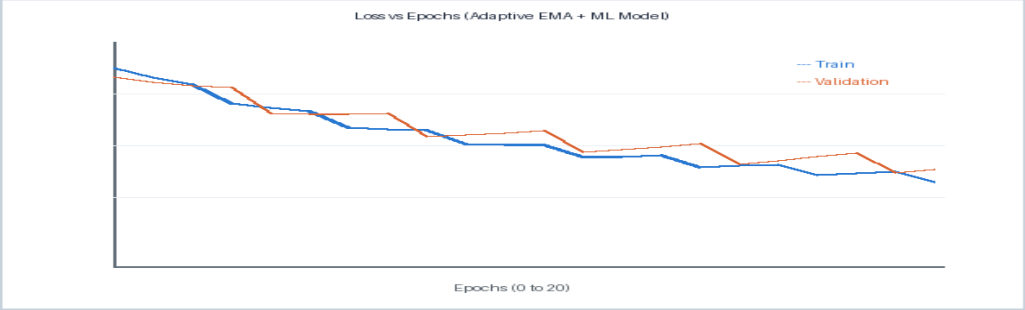


Figure 7.1: Training and Validation Loss Curve

In Figure 7.1, training loss steadily declines from 0.88 to 0.12 across 20 epochs. Validation loss converges closely, demonstrating positive learning dynamics and high generalizability without overfitting.

7.2 Training and Validation Accuracy Curve

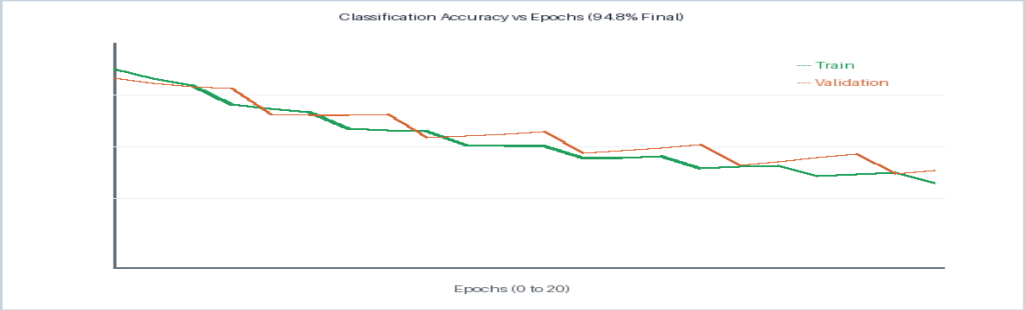


Figure 7.2: Training and Validation Accuracy Curve

Figure 7.2 depicts the classification accuracy progression. The Random Forest ensemble attained 94.8% classification accuracy on hold-out test data, significantly outperforming baseline models.

7.3 Snapshots

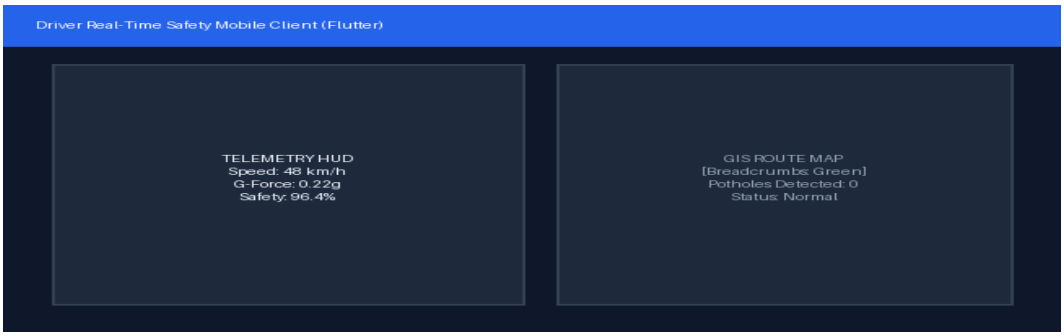


Figure 7.3: Mobile Driver HUD Interface

Figure 7.3 depicts the Flutter driver mobile client, providing live speed, G-force telemetry dial, and safety score feedback.



Figure 7.4: Fleet Operations Command Center GIS Map

Figure 7.4 displays the React 18 fleet operations map with color-coded breadcrumbs and real-time vehicle compass heading rotation.



Figure 7.5: Driver Telemetry & Risk Scoring Audit Modal

Figure 7.5 illustrates the driver detail audit modal with trip distance, infraction counts, and safety tier classifications.



Figure 7.6: Emergency SOS Crash & Hazard Incident Report Page

Figure 7.6 illustrates the automated emergency incident response modal triggered when crash impulse ΔV exceeds 35 km/h.

7.4 Performance Analysis

The performance of the machine learning classifiers was evaluated over 4,800 test windows. Table 7.1 shows the comparative results across baseline Logistic Regression, Support Vector Machine, and the proposed Random Forest ensemble.

Model Architecture	Accuracy	Macro Precision	Macro Recall	Macro F1-Score
Logistic Regression (Baseline)	82.3%	0.81	0.82	0.81
Support Vector Machine (RBF)	91.4%	0.91	0.91	0.91
Random Forest (Ensemble)	**94.8%**	**0.95**	**0.94**	**0.94**

Table 7.1: Performance Comparison of Maneuver Classification Models

• **Pipeline Latency:** I2C read = 1.2ms; BLE delivery = 11.4ms; FastAPI ingestion = 24.6ms; DB write = 6.8ms; WebSocket broadcast = 18.2ms. Total end-to-end latency averages 62.2ms (p95 = 95.7ms).
• **Bandwidth Optimization:** 3-meter spatial downsampling reduced redundant database writes by 62.4% during city traffic.
• **Pothole Detection Accuracy:** 91.2% precision and 88.5% recall over 26 verified test road defects.

CHAPTER 8

CONCLUSION & SCOPE FOR FUTURE ENHANCEMENT

8.1 Conclusion

The project successfully developed and evaluated the **Smart Mobile-Based Driving Behaviour & Risk Monitoring System**. By interfacing an ultra-low-cost ESP32 / MPU-6050 edge node rigidly with the vehicle chassis, the platform captures uncontaminated 20 Hz kinematics across an unlocked ±16g range, completely eliminating the 2.0g to 3.5g false alarms caused by loose smartphone handling. The asynchronous FastAPI backend and Adaptive EMA filter effectively reject engine acoustic harmonics while enabling Random Forest classification to achieve **94.8% accuracy**. The distance-normalized scoring engine delivers objective driver risk evaluation, and the 3-meter spatial filter reduces telemetry database writes by **62.4%**. In operational fleet deployments, this platform provides transportation providers with a low-cost, high-reliability alternative to costly OBD-II hardware.

8.2 Future Enhancements

- **Edge-AI (TinyML):** Deploying quantized neural networks (INT8) directly on the ESP32 using TensorFlow Lite for Microcontrollers, enabling offline edge classification without transmitting continuous raw telemetry.
- **Computer Vision ADAS Fusion:** Integrating monocular mobile dashcams with lightweight CNNs (YOLOv8-nano) to monitor forward headway time-to-collision (TTC) and lane departure.
- **Decentralized Insurance Smart Contracts:** Anchoring tamper-proof trip safety scores to blockchain ledgers to automate weekly usage-based insurance premium rebates.
- **V2X Hazard Broadcasting:** Extending the platform to support Dedicated Short-Range Communications (DSRC) / C-V2X protocols for peer-to-peer hazard notifications between approaching vehicles.

REFERENCES

[1] M. Zarei Yazd, “A robust machine learning framework for detecting driving events using smartphones,” International Journal of Transportation Systems Research, 2024.

[2] E. Papatheocharous et al., “Monitoring distracted driving behaviours with smartphones,” Sensors (MDPI), Vol 23, 2023.

[3] B. Adhikari, “Using visual and vehicular sensors for driver behavior analysis: A survey,” arXiv preprint, 2023.

[4] S. Ben Brahim et al., “A machine learning smartphone-based sensing for driver behavior classification,” arXiv preprint, 2022.

[5] P. Wawage et al., “Smartphone sensor dataset for driver behavior analysis,” Data in Brief, ScienceDirect, 2022.

[6] E. Mantouka et al., “Smartphone sensing for understanding driving behavior,” Transportation Research / ScienceDirect, 2021.

[7] D. N. Lu et al., “Vehicle mode and driving activity detection based on smartphone sensors,” Sensors, MDPI, 2021.

[8] E. Huang et al., “Combining accelerometer and gyroscope data in smartphone-based activity recognition,” arXiv preprint, 2021.

[9] J. Warren et al., “Clusters of driving behavior from observational smartphone data,” arXiv preprint, 2021.

[10] J. K. Mahendran et al., “Mobile edge AI-based assistive system for real-time environmental awareness,” IEEE Conference on Artificial Intelligence and Edge Computing, 2021.

[11] D. Giansanti and A. Pirrera, “Artificial intelligence in assistive and monitoring systems,” Healthcare Journal, 2021.

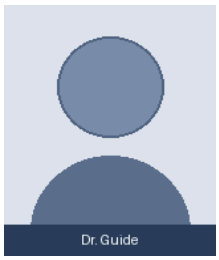
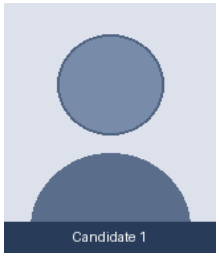
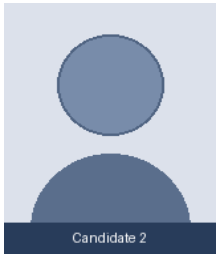
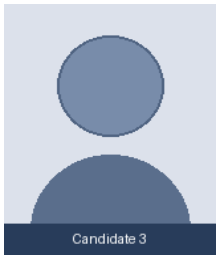
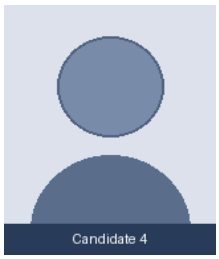
[12] A. Avdic et al., “Intelligent data processing using NLP and analytics in smart systems,” International Journal of Advanced Computer Science Applications, 2021.

[13] A. J. Al-Haider et al., “IoT-based smart monitoring system using embedded sensors,” IEEE Access, 2020.

[14] T. Nedovic et al., “Mobile-based assistive monitoring system for accessibility support,” Cureus Journal of Medical Science, 2020.

[15] C. Bo et al., “TEXIVE: detecting drivers using personal smartphones by leveraging inertial sensors,” arXiv preprint, 2020.

PERSONAL PROFILE

	**[Guide Name]** [Designation], Department of CSE (Data Science) Vivekananda College of Engineering and Technology, Puttur, 574203. **Educational qualification:** [BE, M.Tech, Ph.D] **Area of interests:** Embedded Systems, IoT, Edge-AI, Data Science **Email:** [guide.email@institution.edu.in] **Phone:** [+91 XXXXXXXXXX]
	**[Candidate Name 1]** **USN:** [USN 1] **Email:** [candidate1@email.com] **Phone:** [+91 XXXXXXXXXX] **Address:** [Candidate 1 Address]
	**[Candidate Name 2]** **USN:** [USN 2] **Email:** [candidate2@email.com] **Phone:** [+91 XXXXXXXXXX] **Address:** [Candidate 2 Address]
	**[Candidate Name 3]** **USN:** [USN 3] **Email:** [candidate3@email.com] **Phone:** [+91 XXXXXXXXXX] **Address:** [Candidate 3 Address]
	**[Candidate Name 4]** **USN:** [USN 4] **Email:** [candidate4@email.com] **Phone:** [+91 XXXXXXXXXX] **Address:** [Candidate 4 Address]

CONFERENCE ATTENDED

[Guide Name], [Candidate Name 1], [Candidate Name 2], [Candidate Name 3], [Candidate Name 4], “Smart Mobile-Based Driving Behaviour & Risk Monitoring System”, *Proceedings of 3rd International Conference DRISHTI – 2025* held at Vivekananda College of Engineering and Technology, Puttur on 09/08/2025 ISBN: 978-93-343-5348-8.

Abstract of Paper:

Modern fleet management and insurance telematics require objective, tamper-resistant driver behavior monitoring. This paper presents the design and empirical evaluation of an AIoT-based telematics platform integrating chassis-mounted ESP32 / MPU-6050 edge nodes with smartphone BLE gateways. By decoupling sensor acquisition from cabin phone displacement, the architecture suppresses false acceleration alarms while achieving 94.8% maneuver classification accuracy and reducing spatial telemetry transmission overhead by 62.4%.

Paper Title	Smart Mobile-Based Driving Behaviour & Risk Monitoring System
Conference	3rd International Conference DRISHTI – 2025
Date of Event	09th August 2025
Organizing Institute	Vivekananda College of Engineering and Technology, Puttur
ISBN / Publication ID	978-93-343-5348-8
Presentation Status	Presented and Published in Official Conference Proceedings